

Cluster-based test cases prioritization and selection technique for agile regression testing

Passant Kandil^{*†}, Sherin Moussa and Nagwa Badr

Department of Information Systems, Ain Shams University, Cairo, Egypt

ABSTRACT

Regression testing repeatedly executes test cases of previous builds to validate that the original features are not affected with any new changes. In recent years, regression testing has seen a remarkable progress with the increasing popularity of agile methods, which stress the central role of regression testing in maintaining software quality. The optimum case for regression testing in agile context is to run regression set at the end of each sprint and release, which requires a lot of cost and time. In this paper, we present an automated agile regression testing approach on both the sprints and release levels. The proposed approach addresses both weighted sprint test cases prioritization technique, which prioritizes test cases based on several parameters having real practical weight for testers, and Cluster-based Release Test cases Selection technique that clusters user stories based on the similarity of covered modules to solve the scalability issue. Test cases are then selected based on issues logged for failed test cases using text mining techniques. The proposed approach achieves enhancement for both the prioritization and selection of test cases for agile regression testing. Copyright © 2016 John Wiley & Sons, Ltd.

Received 30 June 2015; Revised 17 January 2016; Accepted 18 May 2016

KEY WORDS: regression testing; agile testing; test selection; test prioritization; clustering; text mining

1. INTRODUCTION

The importance of software testing has grown with the great adoption of agile development methodologies. Agile is a methodology of product delivery, where changes happen rapidly. Hence, the transition of testing practices from the traditional approaches – as the waterfall approach – is taking place with a very high speed [1–3]. This growth can largely be accredited to the end user's expectations and how dynamic the development is in correspondence to the product changes. Regression testing is one of the largest contributors to the overall cost of software development, as it is responsible for a significant percentage of the software maintenance costs, which often dominate the total lifecycle costs [4]. It is performed when changes are made to an existing software. The purpose of regression testing is to provide confidence that the newly introduced changes do not obstruct the behavior of the existing, unchanged part of the software; A complex procedure that is even more challenging with the recent trends in software development paradigms, where companies face time and resources constraints that limit the completion of the testing efforts effectively [5].

Agile methods have brought testing in general, and regression testing in particular, much more into the center of the software development, with their emphasis on techniques such as continuous integration and test-driven development, where some form of testing is taking place continuously [6]. Agile methods encourage frequent builds and test cycles, reducing the intervals between tests [7–10] along with incorporating the practices of user stories and other specifically agile parameters

^{*}Correspondence to: Passant Kandil, Department of Information Systems, Ain Shams University, Cairo, Egypt.

[†]E-mail: passant.kandil@cis.asu.edu.eg

in agile testing to maximize its value for the end-user [11]. Because the value of regression testing is worthy in the agile world with its short milestones, testing teams are always looking for ways to improve the efficiency of their regression test effort to maximally utilize regression testing in an agile context, where test automation and continuous execution should be applied to save cost and time. The target area of the regression test is the complete end-to-end business functions that a system encompasses. The same tests are executed repeatedly over time to ensure that the required behaviors of the system remain stable. In an agile context, the repeatedly execution of test cases means that regression test results are of great value to the product owner at the end of each sprint, as well as the end of the release. There are different types of regression testing techniques, such as (a) test-suite selection, which aims to selecting a relevant subset of test cases from the initial test suite instead of re-running the complete test suite; (b) test-suite reduction, which focuses on finding a minimized set of test cases without compromising fault detection capability; and (c) test-suite prioritization, which schedules test cases to run test cases that are more important earlier so that faults are detected earlier or to provide earlier feedback to testers.

In this paper, we present an agile regression testing approach that is composed of two main techniques. Weighted sprint test cases prioritization (WSTP) technique is a novel prioritization technique for the regression testing of sprints in the agile projects, whereas cluster-based release test cases selection (CRTS) technique is a comprehensive selection technique for the regression testing of releases that selects test cases to use in regression testing at the end of the release in the agile projects. WSTP technique depends on the importance of the requirements represented in the user stories of such agile development. Because the agile software systems are built upon the user stories, certain user stories are more important as customers frequently utilize some features than others. In addition, we emphasize the use of the acceptance test-driven methodology [12, 13], which is based on the communication between the business customers, the developers, and the testers by considering the test cases defined as ‘acceptance test case’, which is testing the acceptance criteria written for the user stories. In addition, WSTP technique takes into consideration the severity of the faults detected from the previous execution of the test cases. CRTS technique clusters the user stories according to the modules covered by each user story, so that user stories covering similar modules – with same functionalities – are grouped into the same cluster. Accordingly, this reduces the required number of pairwise comparisons significantly to solve the scalability problem with the large number of test cases at the release level. Selection of test cases is then carried out based on the features covered by the issues logged on the failed test cases using a text mining technique. The rest of the paper is organized as follows: Section 2 highlights the most relevant work related to the regression testing and agile testing, indicating the main current gaps in this research field. Sections 3 and 4 introduce our new approach for the regression testing of the sprints and releases and our threats to validity for this approach. Section V presents a detailed description of our used data set. Section 6 describes the experiments held, with some screenshots of the implemented system. Sections 7 and 8 discuss and analyze our results, including their implications and the applied evaluation parameters. Finally, Section 9 presents the conclusion and discusses our future work.

2. RELATED WORK

Testing has been a cornerstone of agile software development methodologies since early in the history of the field, particularly the regression testing that has seen a change of status and emphasis with the increasing popularity of agile methods [6, 7, 14, 15]. Different regression testing techniques have been proposed in research [5, 16–18], like test case prioritization and test cases selection. Test case prioritization [19] sorts existing test cases according to certain parameters to attain performance goal and discover faults as earlier as possible, whereas test cases selection [20] selects a relevant subset of test cases from the initial test suite instead of re-running the complete test suit. With the explosion of the data mining techniques usage in the exploration and analysis of large quantities of data, they have been potentially used to solve the automation problems of software testing with large datasets.

Smith *et al.* [21] extended algorithms of selection and prioritization to greedily reduce and prioritize the tests by using both test cost (e.g., execution time) and the ratio of code coverage to test cost. They conducted an experimental analysis on eight case study applications to evaluate the effectiveness of extending the following four techniques: Harrold Gupta Soa [22], delayed greedy, traditional greedy and optimal greedy algorithm, whereas Rani *et al.* [23] introduced a hybrid technique for regression test cases selection for object-oriented programming. This technique was based on both the source code of an object-oriented program and the UML state machine models of the affected classes. The technique was compared with Larsen and Harold's [24], which constructed system dependence graphs for object-oriented software by applying some slicing algorithms. A system dependence graph consisted of a program dependence graph representing the 'main' program either in the system or in a simulation of a calling environment, whereas a class dependence graph represented classes constructed for each class in the system. Yoo *et al.* [25] introduced a clustering technique in which test cases were clustered based on their similarity of the features tested using the runtime behavior. Prioritization of test cases in the same cluster was performed using traditional coverage-based greedy algorithm, where a binary string represented the execution of each test case. Each bit represented a statement in the code. If the test case executes, then the statement corresponding digit is one, else, it is zero. Carlson *et al.* [26] conducted empirical studies using an industrial software product as Microsoft Dynamics X. They implemented another prioritization technique with a clustering approach using code coverage, code complexity, and history data of real faults. This technique used agglomerative hierarchical clustering method, which merged test cases having closer code coverage similarity. The distance between the cluster and any of the remaining test cases was determined by averaging the distance between each element in the cluster and the test case. The test cases were re-ordered, putting the highest average value earlier. In addition, Chen *et al.* [27] introduced a semi-supervised learning technique for regression test selection. This approach introduced a semi-supervised clustering method named semi-supervised K-means, which combined semi-supervised nonlinear dimensionality reduction [28] and K-means. Semi-supervised nonlinear dimensionality reduction was used as a pre-process of the original data with limited constraint information before K-means. The semi-supervised K-means, on the other hand, was used to group tests into clusters. Adaptive sampling strategy was then applied to select data from clusters, where it was evaluated using F-measure.

The previous techniques depend on the code information and the fault history of the prioritization of the test cases. However, they did not take into consideration the requirements of the system, in spite of its importance during the testing phase that has been well understood by the requirements engineering community [29]. The importance of the requirements is critical for the agile projects, where the focus is always on the customer needs and interests. Software systems are built upon the product requirements, having certain requirements are more important than others are. Muthusamy *et al.* [30], on the other hand, introduced another technique for test case prioritization. Their technique assigned six factors for each test case during the test design and analysis phases, which were the customer-allotted priority, code implementation complexity, changes in requirement, fault impact of requirements, and completeness. Prioritization was based on the weighted prioritization value and the weighted priority of each test case in the test suite. Raju *et al.* [31] introduced another cluster-based prioritization technique using the rate of fault detection, requirements volatility, fault impact, and implementation complexity factors entered by the user for each test case. Agglomerative hierarchical clustering was then used to group similar test cases in the same cluster. The prioritization of requirements test algorithm was applied as a value-driven approach to system-level test cases prioritization in order to prioritize traceability between requirements and test cases. The technique measured the importance of testing a requirement by multiplying the factor value for each requirement with the factor weight to evaluate the prioritization factor value. Arafeen *et al.* [32] used requirement-based clustering for test case prioritization. This technique applied text mining in order to determine the clusters of requirements. The clusters of test cases were formed by mapping each test case to its relevant requirements. Prioritization between test cases in the same cluster was performed using the source code information, whereas prioritization between clusters was performed based on the source code information along with the prioritized requests from the client, producing re-ordered test

cases. Ural *et al.* [33] introduced a test case selection technique, considering the coverage of dependencies related to the effects of modification using dependence analysis on an extended finite state machine representation of the requirements of the system under test.

Although the previous techniques depend on the requirements and their importance to apply prioritization techniques, none of them depends on any agile parameter or has ever been applied to agile projects. Arora & Chauhan introduced a test selection technique in the agile environment. They took into consideration the weights of the undirected graph of the user stories group in a module. It is based on the optimal connections in the valued graphs for the test cases selection [34]; they also introduced another test case prioritization method [35] using the noun and verb of a user story to reduce the regression testing time of the test suite after accommodating the new user story test cases. It took into account the nouns and verbs present in the existing user stories and the new user story after doing proper mapping between the new user story's keywords and the existing user stories keywords.

One of the main reasons for the uniqueness of the proposed research is that it applies regression-testing techniques on agile projects, depending on different agile parameters like the business value of the user stories, the test-case type, and the score of issues, having realistic practical weights provided by the tester. Based on our study, these parameters have never been addressed before to tackle this field. It also combines prioritization on the sprints level and selection on the release level to achieve the optimum regression testing for agile projects.

3. THE PROPOSED APPROACH

The target of agile-based regression tests is the complete set (end-to-end) of business functions that a system encompasses. The proposed model aims to provide a fully automated regression testing methodology that prioritizes test cases for regression testing at each sprint of the agile-based projects, and selects test cases to be used in regression testing at the end of the release. It is composed of two main techniques: WSTP and CRTS techniques as described thereafter.

1. WSTP technique

At the sprints level, WSTP prioritizes all test cases for regression testing. The prioritization methodology depends on three main parameters: (a) the business value of the user story; (b) the test case type (either acceptance or functional test case); and (c) the score of issues depending on the count of issues and their severity. For each test case, a value is computed to each parameter separately, and user provides a weight showing the importance of this parameter to calculate the prioritization score. Figure 1a presents WSTP architecture. It consists of five main components as follows:

Component 1: Extract test cases

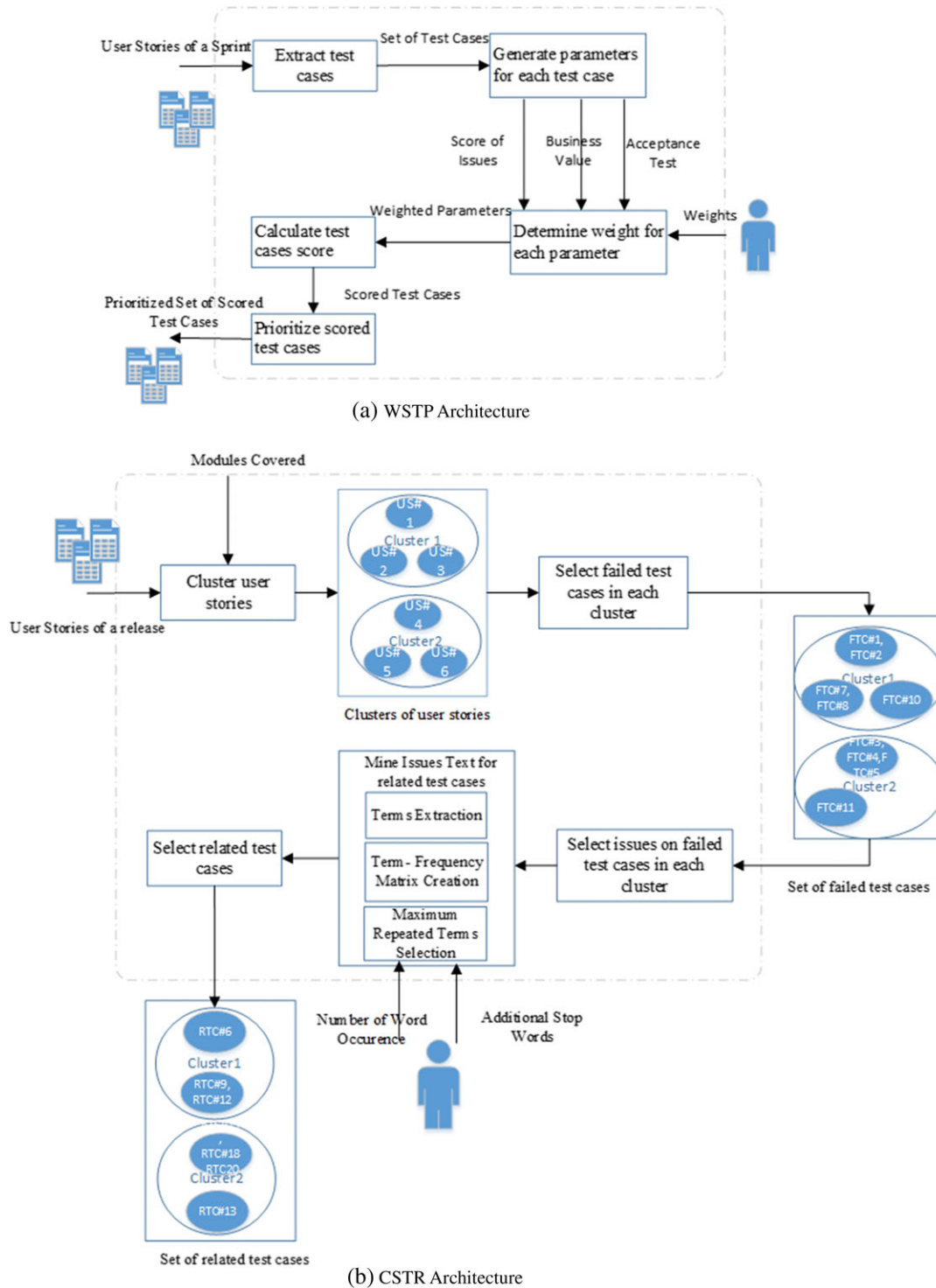
User stories are used in agile software development methodologies as the basis for defining the functions that a system must provide and to facilitate requirements management. A user story is a description consisting of one or more sentences that capture what a user does or needs to do. It represents a small piece of business value that a team should deliver in an iteration. In this component, the user stories for the selected sprint are captured from files, where all the associated test cases are extracted.

Component 2: Generate parameters of each test case

Once test cases are extracted, the following parameters are assigned independently to each test case, as these parameters have no effect on each other.

2.1 Business value of the user story

User stories focus on the customer's value [36]. Therefore, each user story is associated with a business value, which can be assigned by the customer, the development team or by the agreement



(b) CSTR Architecture

Figure 1. Proposed architecture.

of both. In our work, the business value is assigned by the agreement of both the customer and the development team on the value of work; this user story delivers to the customer. A number from 0 to 100 is assigned to the user story during the story writing; the greater the number, the higher the value it is to the customer.

2.2 Test case type

User stories are associated with many test cases, where each test case is identified as one of the following two types:

- 1- *Acceptance test case*: These tests typically cover the acceptance criteria of the user story (i.e., the high-level test scenario). The acceptance criteria provide the definition of 'Done' for the story. Many acceptance criteria can be listed in order to clarify the intent of the story. These acceptance criteria should be proven through a set of test cases defined as acceptance test cases, which prove that the product delivers the required business value to the customer.
- 2- *Functional test case*: These are the test cases written for the different scenarios of the user story, covering all the possible scenarios that the tester needs to check.

If the test case is identified as an acceptance test case, it should have a higher priority than the other functional test cases. Therefore, if the test case is of type 'acceptance test case', it is set to '1'; otherwise, it is a functional test case and is set as '0'.

2.3 Score of issues

This parameter not only considers the count of issues detected from the previous execution of each test case but also the priority of the issues that were determined by the team when logging the issue. It is unlikely to treat all issues like each other, that is, the issue logged with priority 'emergency' cannot be considered with the same importance as the issue logged as 'low'. The score of issues is evaluated using the equation later, where priority score is defined as shown in Table I.

$$\text{Score of issues for each test case} = \sum_{i=1}^n \text{count of (issue } i) * \text{priority } i \quad (1)$$

where n is the number of priorities classes.

Component 3: Determine Weight for Each Parameter

In this component, a weight is given for each of the above parameters indicating how important the parameter is for calculating the prioritization technique. The system user, i.e. the tester, provides the weights, where the sum of the given weights should be 100%.

Component 4: Calculating Test Cases Score

A score for each test case is then computed from the parameter value and the parameter weight using the following formula:

$$\text{Score of test Case} = (BV * WBV) + (CT * WCT) + (SI * WSI) \quad (2)$$

where BV is the business value of the user story, WBV is its weight, CT is the test case type (either acceptance or functional) evaluated to '1' or '0' respectively, WCT is its weight. SI is the value of score of issues calculated in Equation (1), and WSI is its weight. All weights are entered by the tester.

Component 5: Prioritize scored test cases

Based on the previously illustrated methodology, the prioritization technique is applied depending on the score given to each test case. Test cases are then ordered descending for execution according to the score values; test cases with the higher score will be given a higher priority. This assures that the higher priority test cases run first in the regression-testing phase. The final output of the proposed technique supports the testing team with a set of prioritized test cases for each sprint in the agile development process.

Table I. Priorities Score.

Priority	Priority Score	Priority	Priority Score
Ship Stopper	3	Medium	1.5
Emergency	2.5	Low	1
High	2	Very Low	0.5

2. CRTS Technique

As for the release level, CRTS technique selects the test cases to execute at the regression testing of the release. In order to deal with the large number of test cases at the release level, a clustering technique is applied to cluster the user stories according to the similar functionalities they cover. Hence, dealing with clusters of user stories instead of single ones reduces the number of pairwise comparisons. Failed test cases are then selected from a single cluster. CRTS not only considers the failed test cases but also the related test cases, because regression testing is mainly for the affected functionalities from previous failures. The related test cases are determined by applying text mining to the issues on the failed test cases in each cluster. It finds the most frequent terms in those issues, then it selects the test cases containing these terms. The resulting set of test cases represents all the test cases that test the functionalities that previously exposed those issues. Figure 1b presents CRTS architecture, having five main components as described thereafter.

Component 1: Cluster user stories

Programs/applications are composed of one or more developed modules, where each module represents a piece of code that serves some business operations. In this component, the user stories of the release are clustered according to the modules covered by the user stories, so that stories covering similar functionalities are grouped in the same cluster. This reduces the required number of pairwise comparisons significantly. Thus, it solves the scalability problem of other algorithms with large-scale test cases data. The clustering K-means method is applied [37], where the total number of clusters k is the number of modules in the application. The user stories are represented as a string of zeroes and ones. The distance between a user story and the centroid of a cluster is calculated using the hamming distance [38], that is, the count of bits that are different in the two strings. Hamming distance is calculated by creating a sequence of Boolean values, indicating the mismatches and matches between corresponding positions in the two inputs. It then sums the sequence with False and True values interpreted as zeroes and ones. The algorithm returns the mean/center, for each cluster, with each user story assigned to the specific cluster having the least distance to it.

Component 2: Determine failed test cases and issues related to them in cluster

Based on the clusters of user stories created in the previous component, a selection technique is applied to define the failed test cases from each cluster and the generated issues with them. The selection of failed test cases and their related issues occurs from each cluster separately, which allows dealing efficiently with large datasets of test cases by reducing the number of pairwise comparisons between test cases for selection. Thus, it makes the proposed approach scalable to regression testing with large datasets concerns. For each cluster, the failed test cases are the first selected set of test cases for regression in the proposed algorithm.

Component 3: Text mining for related test cases

The selection of failed test cases is not enough for regression testing, as regression testing is carried out to ensure that changes made are not affecting the previously working functionality. Accordingly, the proposed algorithm determines the related test cases that were affected by the changes done. Related test cases can be defined as the test cases that were not determined as “failed” but they test the same functionalities of the failed ones. Thus, they are probably the most suggested test cases that may be affected with the changes. Hence, the proposed algorithm defines the related test cases using the issues occurred from the failure of the test cases execution, where fixes have been applied to errors discovered by the failed test cases. In this component, a text-mining technique is applied on the defined issues to obtain the most frequent repeated terms in these issues. This is used to determine the related test cases to the most failures happened in the system.

3.1 Terms extraction

In this process, words are extracted from each issue (i.e., considered as a set of words or terms). For better accuracy of the selection technique, the system user, that is, tester, can add additional stop words, in addition to the normal list of stop words. These words represent the repeated words in the issues with

no significance to identify the related test cases, as they do not represent a failure in the system functionality. An example of extra additional words is the general words in software environment, such as user and system or could be words related to the specific domain of the tested application. After eliminating those words, a list of all distinct terms is obtained across all the issues in each cluster.

3.2 Term-frequency matrix creation

All distinct terms obtained from the previous step are used to create a term-frequency matrix. In this matrix, each term is associated with its number of occurrences in the issues of each cluster.

3.3 Maximum repeated terms selection

After obtaining the repetition frequency of each term in the previous process, the terms having their frequency exceeding a user-given threshold are selected. Those terms represent the important terms to consider in selection.

Component 4: Generate related test cases

In this component, the test cases containing the maximum repeated terms in the previous process are selected per each cluster. These test cases are considered to be the related test cases that test the functionalities invoking the issues, and most likely to be affected by the changes. Hence, the selected test cases are the most important at the regression phase. This resulting list of test cases, along with the failed test cases, is the final set of test cases extracted to the system user to use for the release regression testing.

4. THREATS TO VALIDITY

Considering the internal validity of the proposed approach, by discussing the basis on which we determined the parameters to be used in the proposed techniques. By studying the agile framework, we found that agile projects consist of user stories that deliver small and independent chunks of values; each user story must be accompanied by (a) acceptance criteria defining when to accept this story, (b) the business value that defines importance of each user story from the business prospective, and (c) the story points defining the estimated effort for implementing the user story. In the proposed research study, the main fundamentals of the agile framework that affect the regression testing are evaluated, like the existence of acceptance test cases for the acceptance criteria and the business value of the user story. The story points are not investigated in this study, as we are currently concerned with the optimum list of test cases to run in the regression phase rather than the development effort associated with the user stories. Regarding the external validity, we applied our experiments on three data sets only for three different real systems from diverse domains and with variant sizes. As for the construct and statistical validity, we compared our system results with the actual values derived from the testing team's execution to the different datasets as presented in Section 7.

5. DATASETS

For our experiments, we used three datasets representing three different system releases. The datasets come from different domains with variant sizes as shown in Table II. These datasets have been prepared as a set of files for each release as shown in Figure 2 and described thereafter.

Table II. Data Sets Sizes.

	Release 1	Release 2	Release 3
No. of Sprints	11	11	12
No. of User Stories	45	28	30
No. of Test Cases	359	888	549
N of Issues	173	197	40

Sprint_ID	Sprint Date
Sprint01	14/06/2014
Sprint02	29/06/2014
Sprint03	07/07/2014
Sprint04	15/07/2014
Sprint05	30/07/2014

(a) Sprints File

Story_id	Story_title	Story Point	Business Value	Sprint Id	Module Name
1	Add Structure Item	5	30	Sprint01	Structure Mgmt
2	Delete Structure Item	6	30	Sprint02	Structure Mgmt,KPI Wizard
3	Define KPI attribute of type dimension	7	10	Sprint 10	KPI Wizard,Structure Mgmt,Dashboards Features
4	GPM Lite KPI Wizard	10	80	Sprint04	KPI Wizard
5	List Dimensions webpart	11	20	Sprint05	KPI Wizard,Structure Mgmt,Dashboards Features
6	Manage Dimension members	15	10	Sprint02	KPI Wizard
7	Delete KPI	14	10	Sprint04	Structure Mgmt,Dashboards Features

(b) User Stories File

Testcase Id	Testcase Title	Acceptance Test Case	User_Story_id
1	Check folders of metrics displayed	Acceptance Test Case	1
2	Check that for each metric folder from folders list, correct metrics list are displayed in metrics' list	Acceptance Test Case	1
3	Check that there is a textbox that enables user to write its formula in	Test Case	1
4	Check that for each operand in KPI formula, it is listed in symbols list and an empty field is put to add metric in it	Test Case	1
5	Check that dragging and dropping metric is working correctly	Test Case	1
6	Try updating metric name or metric folder name and check names in creating KPI	Test Case	1
7	Try deleting metric folder or metric and check their lists in KPI wizard	Acceptance Test Case	1

(c) Test Cases File

Issue_Id	Issue_title	Testcase_id	Priority
1	While adding new KPI with formula and select its operands than delete the formula , no validation message appear	3	High
2	Javascript error occurs if user is creating KPI with formula and went to the previous step while the selected folder is not the first one	3	Medium
3	JavaScript error is displayed in creating KPI with formula while KPI threshold values are not entered	4	Emergency
4	The validation messages on set formula appear beside each others	3,4	Low
5	Java script error always appear in KPI wizard in case the calculation method is formula	4	Medium
6	System posts back to display an error message validation, while it should be client side validation	10	Emergency

(d) Issues File

Figure 2. Files structure.

- 1- *Sprints File*: Containing the sprints in each release. The number of each sprint and its date are captured as shown in Figure 2a.
- 2- *User Stories File*: Containing the user stories of the release. For each user story, the following data is captured: the story number, the story title, the story points (representing the size of the user story), and the business value (representing the importance to the customer), the sprint that the user story is related to, and the module name that this user story covers. Figure 2b shows a sample of a user stories file.
- 3- *Test Cases File*: Containing the test cases of the release. The test case number, its title, the test case type (whether it is an acceptance or functional test case), the user story that the test case tests) are captured for each test case as shown in Figure 2c.
- 4- *Issues File*: Containing the issues from the previous execution of the test cases. For each issue, the following data are captured: the issue number, its title, the test case (that the issue was entered on), the issue importance, either 'Ship Stopper-Emergency-High-Medium-Low-Very Low'. Figure 2d shows a part of an issues file used.

6. EXPERIMENTS

1. WSTP Experiments

Weighted sprint test cases prioritization technique applies three parameters to calculate the score of each test case: Customer value, test case type (acceptance or functional), and score of issues. To understand how the testing team prioritizes the test cases using these parameters in real applications, we launched a small survey containing a sample set of test cases with variant values for these three parameters, the survey investigated the criteria used for the prioritization and the manual time required for the prioritization and selection of test cases for the regression phase. The survey was sent to 10 agile testing experts with different testing experiences who work on various agile projects at one of the largest IT companies in Cairo, Egypt. Figure 3a shows the survey content having the sample set of test cases, whereas Figure 3b–d represents samples of the received replies. It is concluded, through the survey replies, that the testing team prioritizes the set of test cases according to different importance given to the three parameters. The importance may vary according to the sprint characteristics like the business value of the user stories in the sprint, the type of test cases and the issues logged on them or according to the situation regression is needed to run, where the execution order of the test cases may change. To model the real-life practice of regression testing in agile projects, the proposed algorithm applies a weight for each of the aforementioned parameters.

Question :

If we have to run regression testing on a sprint containing the below test cases,

- 1-How will you order the below test cases to run the most important first having
 - a. Customer value of the user story that the test cases written for .
 - b. If it is an acceptance test case or No.
 - c. Number of issues on each test case with Priority of the issues.
- 2- what is your criteria in ordering
- 3- What is the average time you need to manually prioritize a test case set with an average of 60 test cases
- 4- What is the average time you need to manually select specific test cases from a test case set with an average of 600 test cases

	Customer Value of the user story	Acceptance Test Case	No. of Issues	Priority of Issues
Test Case1	20	Yes	3	2 High , 1 Medium
Test Case2	40	No	3	1 Emergency, 1 High, 1 Medium
Test Case3	40	Yes	2	2 High
Test Case4	60	No	3	1 Ship Stopper , 1 Emergency, 1 Medium
Test Case5	50	Yes	4	1 High, 2 Medium , 1 low

(a) Survey Question

Answer :

- 1-Order of Test Cases : 5,3,1,4,2
- 2-Criteria : the acceptance test cases are the most important test cases then order by customer value. I only have a comment on test case 4 that according to the situation the order of this test case should be executed.
- 3- Prioritization Time for 60 test cases : 10 minutes
- 4- Selection Time for 600 test cases : 30 minutes

(c) Survey Reply 1

Answer :**1-Order of Test Cases .**

- a) Test case4
- b) Test case5
- c) Test case2
- d) Test case3
- e) Test case1&3 are almost with the same importance

2-What is your criteria in ordering the test cases for regression.

- Number of issues and its priority As per sprint we need to have regression on the stories that have the most fixes as they are the ones that would have probabilities that fixes affected other parts
 - Acceptance or not
 - Business value has the least priority through sprint as already all stories selected for sprint should be finalized
- 3-Average time needed for prioritization : 15 minutes
4-Average time needed for Selection:40 minutes

(b) Survey Reply 2

ANSWER :

- 1- Order of Test Cases .
Order is: 4, 5, 2, 3,1

2- What is your criteria in ordering the test cases for regression.

I couldn't find a concrete criteria in prioritizing, but I can tell you my explanation :

- TC4 has a high customer value with 1 ships topper & 1 emergency
 - TC5 is an acceptance with high customer value than with 1 high issue
 - TC2 is not an acceptance but has 1 emergency and 2 high
 - TC3 Acceptance and high customer value than TC 1 (both has 2 high issues)
 - And finally TC1
- 3- Manual average time for prioritization :10 minutes
4- Manual average time for selection :40 minutes

(d) Survey Reply 3

Figure 3. The survey question and samples of replies.

The weight represents the importance of the parameter while prioritization. Therefore, each parameter is a given a weight from zero to 100, having the sum of weights of the three parameters equals to 100. To study the effect of the different weights applied for the three different parameters, we applied in our experiments the five values [10, 30, 50, 70, 90]; as by experiments, these values almost represent the different ranges that could be applied for each parameter. We applied the set of ranges for each of the parameter on the three different datasets. Values for each weight are applied on 12 different sprints with variant characteristics, where four sprints are selected from each release to apply the weights on.

2. CRTS Experiments

The following steps were applied in our experiments:

1. For each cluster, the issues on the failed test cases are selected. A sample of the issues on some failed test cases in a cluster is presented thereafter:

Issue 1 KPIs with formula are not updating correctly in PPS

Issue 2 If there are more than validation, event system displays metrics first

Issue 3 Maximum length validation is not applied on metrics' updating

Issue 4 System does not post back to display message validation for actual values

Issue 5 Metrics are invisible while dragging it in "Metrics" text box

2. From the selected issues, the distinct terms are extracted and then the stop words and the additional stop words are removed automatically. Example: additional stop words like 'system' and 'user' are added, where the term-frequency matrix is created as shown in Table III.
3. The repeated terms are selected according to a provided threshold for the number of word occurrences. Example: if the provided threshold for the number of word occurrences is 1, then the repeated words that will be selected from Table III are 'Updating', 'Validation' and 'Metrics', whereas if this threshold is 2 or 3, the selected repeated words will be Validation and Metrics.

Table III. Term-Frequency Matrix

Term	Frequency	Term	Frequency	Term	Frequency	Term	Frequency
KPI	1	PPS	1	First	1	Applied	1
Formula	1	Validation	3	Maximum	1	Post	1
Updating	2	Event	1	Length	1	Back	1
Correctly	1	Display	1	Dragging	1	Display	1
Text	1	Message	1	Metrics	3	Values	1
Box	1	Actual	1	Invisible	1		

4. The test cases in the cluster containing the repeated words are selected, indicating that they test the same functionalities that exposed the issues. Example: in case of the defined threshold for the number of word occurrences is 1, the selected related test cases are

[For the repeated, word 'Updating']

Test case 5 Try updating metric folder name and check names in creating KPI

[For the repeated, word 'Validation']

Test case 4 View Actual Values validation will appear if metric does not have data source defined yet

[For the repeated, word 'Metrics']

Test case 1 Search for metrics that exists in a folder that the user has viewer access

Test case 2 Check that User has the ability to change the linked metrics

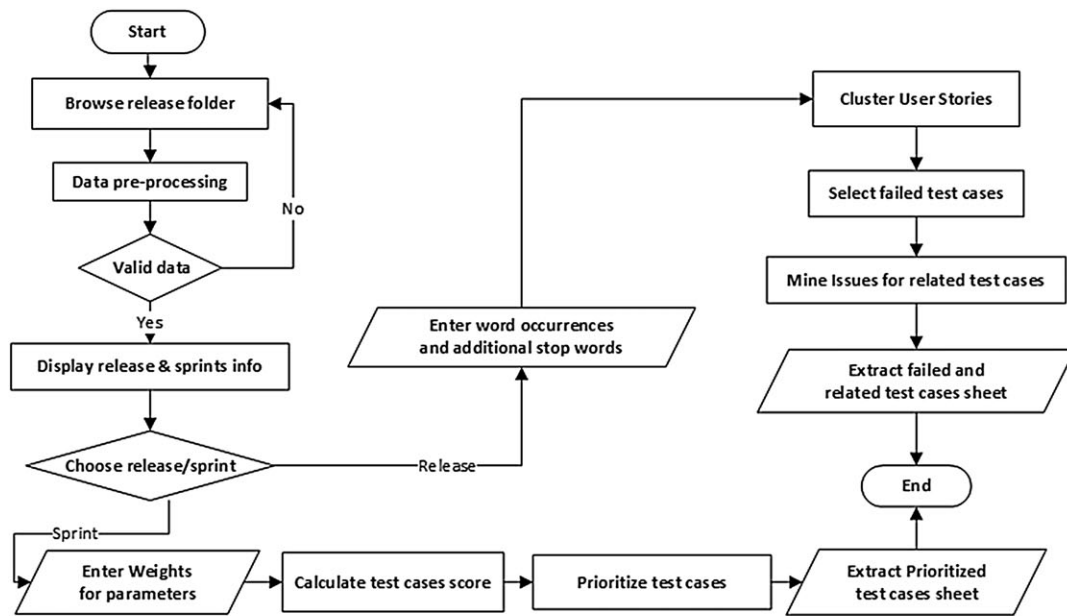
Test case 3 Metrics Viewer User should not be able to see option "Manage" from Actions menu in the Metrics Folders section

In case of the defined threshold for the number of word occurrences is 2 or 3, all of the aforementioned test cases are selected, except for test case 5, because the repeated word 'updating' is below the threshold.

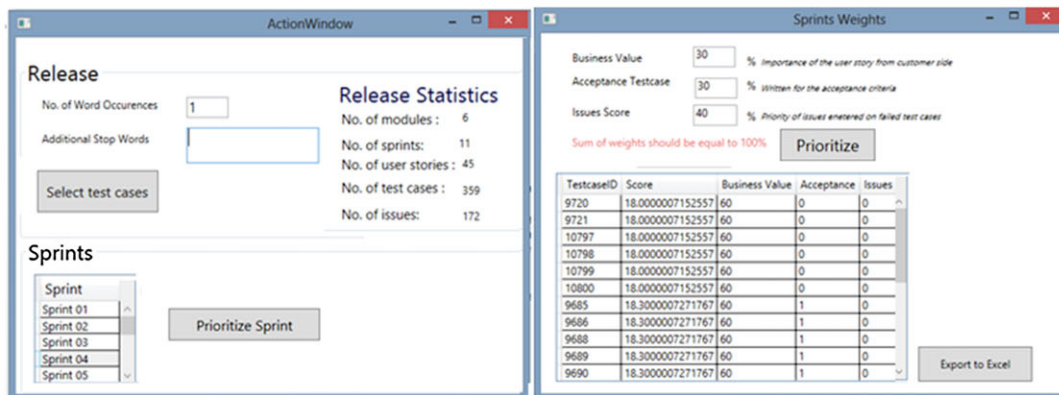
3. Implemented system and screens

The system flowchart for the presented approach is presented in Figure 4a, which describes the system flow as follows:

1. The system user uploads the release folder containing the files as described in Section 4.
2. System pre-processes the datasets in the files to validate the structure and content of the uploaded folder. It checks no duplicate records in any file and that data integrity exists, where data should be consistent and related to each other.
3. System then displays the statistics of the uploaded release as shown in Figure 4b, showing the number of modules in the release, the number of user stories, the number of test cases and the number of associated issues. All the release sprints are also displayed.
- 3.1 In case of applying regression testing to a sprint:
 - 3.1.1 The system user determines the required sprint, the required weight for each of the parameters, and then he prioritizes.
 - 3.1.2 System calculates the score of each test case and prioritizes the test cases based on the highest scores.
 - 3.1.3 The prioritized test cases are generated having the most important test cases first as in Figure 4c.
- 3.2 In case of applying regression testing to a release:
 - 3.2.1 The system user provides the threshold of the number of word occurrences, with any additional stop words, and then select test cases.
 - 3.2.2 System clusters the user stories and then it selects the failed test cases and the associated related test cases through mining the issues on the failed test cases. Each selected test case is presented by the cluster it belongs to, and whether it belongs to the failed set or the related test cases set, where 'Succeeded' is set to false in case it belongs to the failed test cases, otherwise, it is set to



a. System Flow Chart



b. Action Window

c. Sprints Weight Window

Failed Test Cases: 38.44%, Selected Test Cases: 85.52%

Test Case ID	Cluster #	Succeeded	Keyword(s)
10960	0	False	grid empty
10966	0	False	
11129	0	False	
11131	0	False	
11157	0	False	kpi wizard access metrics
11517	0	False	
11518	0	False	
11520	0	False	metric
11521	0	False	metric
11680	0	False	metric
10932	0	True	metric
10935	0	True	metric
10939	0	True	metric
10945	0	True	metric
10948	0	True	metric
10957	0	True	grid

d. Release Results Window

Figure 4. System flowchart and the associated screenshots.

true to differentiate between the failed test cases and the selected ones from the text mining. Finally comes the keywords identified for each test case as shown in Figure 4d.

Prioritization Pseudo Code

```

Get BVWeight, CTWeight, ISWeight
Int user_stories_id [] = Select_user_stories (sprint_id);
For (int i = 1 to i = n) // Loop on all user stories
    Int test_cases_id [] = Get_test_cases (user_stories_id[i]);
    For (int j = 1 to j = k) // for all test cases of the current user story
        GetBV_CTtest_cases_id[j];
        If CT = "Acceptance Test"
            Set CT = 1
        Else Set CT = 0
        EndIf
        Set IS = 0
        For (int y = 1 to y = m) // for all issues for the current test case
            If Priority[j][y] = "Ship Stopper"
                IS += 3.0;
            Else if Priority[j][y] = "Emergency"
                IS += 2.5;
            Else if Priority[j][y] = "High"
                IS += 2.0;
            Else if Priority[j][y] = "Medium"
                IS += 1.5;
            Else if Priority[j][y] = "Low"
                IS += 1;
            Else IS += 0.5;
            EndIf
        EndForLoop
    Set TCS[j] = (BV[j] * BVW) + (CT[j] * CTWeight) + (IS * ISW)
EndForLoop
Prioritize_TCs (TCS[j]);
EndForLoop

```

Selection Pseudo Code

```

Get No_Word_Occurences, Stop_Words
Int user_stories_id [] = Select_user_stories (release_id);
String modules [] = Select_Stories_modules (user_Stories_id []);
Cluster_Stories_Clusters [] = Cluster_user_stories (user_stories_id [], modules []);
For (int i = 1 to i = n) // for all user stories clusters
    Int failed_test_cases [] =
    Select_FailedTcsPerCluster (clusters[i]);
    String issues [] = Select_Issues
    (Failed_test_cases [i]);
    For (int j = 0 to j = m) // for all issues in the cluster
        String keywords [] = Get_Keywords (issues[i] [j]);
        Int frequency = Calculate_Frequency (keyword[i] [j]);
        If Frequency > No_Word_Occurences
            Int related_test_cases [] =
            Select_RelatedTcsPerCluster (keyword[i] [j])
        EndIf
    En EndForLoop r
EndForLoop Loop
Return failed_test_cases, related_test_cases

```

Table IV. WSTP Experimental Results

	APFD	10	30	50	70	90	APFD	10	30	50	70	90
Release1 Sprint4	BV	0.8	0.93	0.93	0.93	0.93	Release2 Sprint8	BV	0.82	0.84	0.84	0.82
	CT	0.73	0.8	0.8	0.8	0.47		CT	0.79	0.82	0.82	0.74
	SI	0.8	0.92	0.93	0.93	0.93		SI	0.82	0.84	0.84	0.84
	Avg.ET	10 seconds						Avg.ET	11 seconds			
Release1 Sprint5	BV	0.62	0.73	0.81	0.81	0.81	Release2 Sprint10	BV	0.66	0.65	0.65	0.66
	CT	0.79	0.81	0.81	0.81	0.63		CT	0.65	0.65	0.65	0.62
	SI	0.81	0.81	0.83	0.83	0.83		SI	0.65	0.66	0.67	0.72
	Avg.ET	53 seconds						Avg.ET	47 seconds			
Release1 Sprint6	BV	0.53	0.54	0.55	0.55	0.55	Release3 Sprint2	BV	0.69	0.85	0.85	0.85
	CT	0.56	0.55	0.54	0.54	0.39		CT	0.69	0.85	0.85	0.47
	SI	0.54	0.55	0.56	0.56	0.56		SI	0.85	0.85	0.85	0.854
	Avg.ET	24 seconds						Avg.ET	6 seconds			
Release1 Sprint8	BV	0.77	0.71	0.74	0.70	0.69	Release3 Sprint4	BV	0.75	0.75	0.75	0.75
	CT	0.74	0.74	0.74	0.74	0.74		CT	0.75	0.75	0.75	0.75
	SI	0.74	0.74	0.86	0.86	0.86		SI	0.72	0.75	0.75	0.75
	Avg.ET	22 seconds						Avg.ET	9 seconds			
Release2 Sprint2	BV	0.74	0.82	0.75	0.75	0.752	Release3 Sprint11	BV	0.76	0.86	0.86	0.84
	CT	0.83	0.75	0.72	0.74	0.61		CT	0.86	0.86	0.85	0.79
	SI	0.72	0.82	0.82	0.83	0.83		SI	0.86	0.84	0.86	0.974
	Avg.ET	10 seconds						Avg.ET	4 seconds			
Release2 Sprint5	BV	0.81	0.89	0.89	0.89	0.89	Release3 Sprint12	BV	0.97	0.97	0.97	0.97
	CT	0.74	0.81	0.81	0.81	0.51		CT	0.97	0.97	0.97	0.97
	SI	0.74	0.89	0.89	0.89	0.89		SI	0.97	0.97	0.97	0.97
	Avg.ET	8 seconds						Avg.ET	6 seconds			
10												
	Computed	Actual	Computed	Actual	Computed	Actual	Computed	Actual	Computed	Actual	Computed	Actual
BV	0.74	0.75	0.80	0.80	0.80	0.80	0.79	0.80	0.79	0.80	0.79	0.80
CT	0.76	0.78	0.78	0.79	0.77	0.75	0.78	0.75	0.64	0.65	0.64	0.65
SI	0.77	0.75	0.8	0.8	0.82	0.79	0.83	0.81	0.82	0.81	0.82	0.81
90												

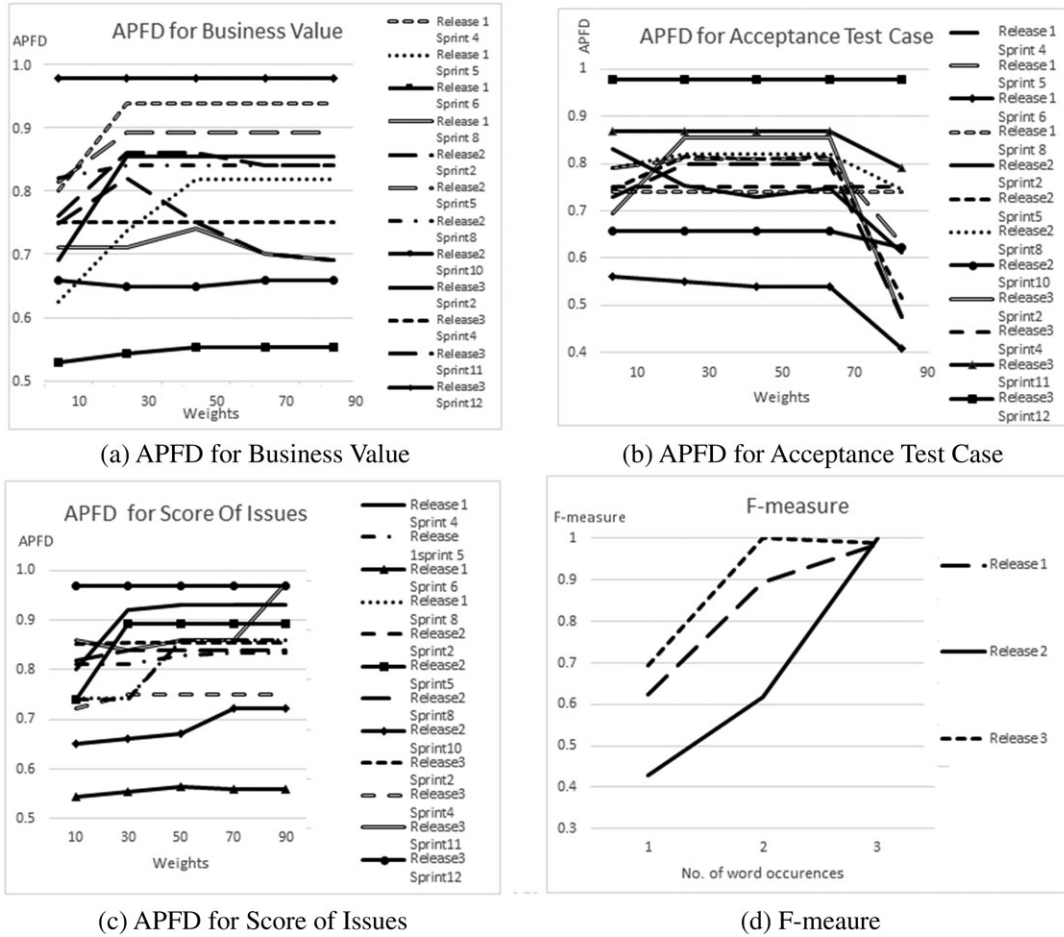


Figure 5. Charts.

7. RESULTS AND EVALUATION PARAMETERS

1. WSTP Results and evaluation

Average percentage fault detection (APFD) is used to evaluate the proposed WSTP technique. APFD provides values ranging from 0 to 100, indicating how successful the prioritization technique is; the closer the value to 100, the better the prioritization technique is [5, 39, 40]. The APFD for test suite T is given by the following equation:

$$APFD = 1 - ((TF_1 + TF_2 + TF_i \dots TF_m) / nm) + 1/2n \quad (3)$$

where TF_i represents the i th position of the test case in the test case suite T that exposes faults, m is the total number of faults exposed in the system/module under T, and n is the total number of test cases in T.

Table IV shows the experimental results for WSTP algorithm using the different weights of the three parameters. The first column in the table shows the selected sprint from each release, whereas the subsequent columns show the weights given to the parameters to calculate APFD, which represents the fault detection capability along with the average execution time (Avg. ET) of the system. Figure 5a–c shows APFD variation for the three parameters through the different applied weights; the second part of the table shows the comparison between the averages of APFD computed for each parameter with the different weights used to the average of the actual APFD computed. Studying the results, we found that APFD varies with the different weights used depending on the characteristics of the sprints. In case the sprint has the same value for a parameter for all the user stories in it, then APFD will not be different for those sprints, as in release 3 sprint 12 and release 3

Table V. Release Results

Releases	Release 1			Release 2			Release 3		
Word Occ.	1	2	3	1	2	3	1	2	3
No. of Failed	138	138	138	192	192	192	36	36	36
No. of Selected	305	170	142	703	431	192	68	36	36
F-measure	0.62	0.89	0.98	0.42	0.61	1	0.69	1	1
Avg. Execution Time	50 seconds			1 min., 50 seconds			1 min., 20 seconds		
Word Occurrences	1			2			3		
F-measure Computed	0.57			0.833			0.993		
F-measure Actual	0.6			0.8			0.85		

sprint 4. The APFD value remains the same for the different weights used for the business value (WBV), as all the user stories of those sprints have the same business value. Therefore, the weight given for the business value parameter did not affect APFD measure. In release 1 sprint 8 and release 3 sprint 12, APFD remains the same for different weights used for the test case type (WCT), as no test cases of type ‘acceptance’ are found in those sprints. In case of release3 sprint12, APFD remains the same for the different weights used for the score of issues (WSI), where no issues are found for the test cases of those sprints. For all other sprints, the maximum APFD is reached when the weight of the business value (WBV) is 30 or 50 and when the weight of test case type (WCT) is 30, 50 or 70, while APFD decreases dramatically with some sprints when the weight of the test case type (WCT) is 90. Therefore, it is not recommended to use a very high weight for the acceptance test case. The maximum APFD is reached when the weight of the score of issues (WSI) is 50 or 70, which indicates that this is the optimal range of weights to be used for the different parameters to obtain best results. Our results show that the calculated average APFD for the different business value weights equals to 0.78, for the different acceptance test case equals to 0.75, and for the different score of issues equals to 0.81. Thus, the average fault detection capability (APFD) of our proposed approach for the different parameters equals to 0.78 with average execution time of the system equals to 18 s, which is very minimal compared with the average actual required time that is equal to 12 min calculated as per the survey replies in Figure 3.

2. CRTS Results and evaluation

F-measure is used to evaluate the proposed system. It evaluates the integrative benefit of the precision and recall measures by the combination of the two parameters [27].

$$F_Measure = (2 * Precision * Recall) / (Precision + Recall) \quad (4)$$

Precision represents the accuracy degree of selection, which measures the extent to which a selective re-test strategy S ignores non-modification-revealing test cases. Recall represents the completeness of test selection, which measures the proportion of selected failed tests from all failed tests.

$$Precision = |T'F|/|T'| \quad (5)$$

$$Recall = |T'F|/|T| \quad (6)$$

where $T'F$ is the set of failed test cases from T' , which is the set of selected test cases.

For CRTS technique evaluation, we applied different number of word occurrences on each of the three releases datasets then analyzed the results associated with the number of word occurrences equals to 1, 2, 3, 4... in order to determine the suitable threshold for the word occurrences. As per the experiments applied, no related test cases are selected when the number of word occurrences increases than three words. Accordingly, results are evaluated for only three different values of word occurrences, that is, 1, 2, and 3, because all word occurrences – that were more than three – selected failed test cases only with no related test cases. F-measure is computed for the threshold of number of word occurrences equal to 1, 2 and 3 for each of the three different releases. Table V first part

shows the number of failed test cases, total selected test cases and the F-measure calculated for each release for the different number of word occurrences along with the execution time of the system, while second part shows the comparison of the F-measure computed with the actual F-measure. Figure 5d shows F-measure variation for the different parameters. Studying the results, the number of the selected test cases is inversely proportional to the threshold of the number of word occurrences. The results show that the calculated average of F-measure evaluated for the different releases with different number of word occurrences equals to 0.79. The average execution time of the system is 1 min and 30 s, which is minimal compared with the average actual required time that is equal to 36 min calculated as per the survey replies in Figure 3.

8. DISCUSSION OF RESULTS

The proposed techniques present a complete framework for regression agile testing on the sprints and release levels. The first proposed technique WSTP prioritizes test cases on the sprint's level using agile-defined parameters and weights from the users. The second proposed technique CRTS selects test cases on the release level, in addition to the failed test cases and any related test cases that test the affected functionalities from previous failures. CRTS uses a clustering approach to deal with the large number of test cases on the release level accompanied with the large-scale systems. Thus, it handles the scalability concerns that can be faced during regression testing with such large-scale systems, as it decreases the number of pairwise comparisons between test cases. Both techniques complement each other to produce a complete model for agile regression testing on all the levels. Interpreting WSTP's results, our technique produces an average of APFD for the different parameters weights equals to 0.78. This means that our prioritization technique improves the fault detection capabilities by 78% (i.e., faults are detected earlier), having the closer APFD to 100% the better the prioritization technique is. In addition, comparing our computed values to the actual ones indicates very close results. Interpreting CRTS's results as shown in Table IV, our technique produces an average F-Measure equals to 0.79 for the different number of used word occurrences. This means that the integrative benefit of the precision and recall measures representing the accuracy and completeness of test cases selection is 79%, where the closer F-measure to 100%, the better the selection technique is [27]. Comparing our calculations with the actual values provides very close results as shown in Table V. We could not compare our results with other techniques, as to the best of our knowledge; we could not find other techniques applied similar agile parameters on agile projects for test cases prioritization and selection at the regression phase.

9. CONCLUSION AND FUTURE WORK

In this paper, we introduced an approach for automated agile regression testing that is composed of two main techniques: WSTP and CRTS techniques for agile regression testing at the sprints and release levels, respectively. Although different studies have been conducted to regression testing, very little attention is given to using the parameters defined specifically for the agile projects. The proposed techniques have used essential parameters specifically for the agile projects, such as the business value, the test case type, and the score of issues to improve the regression testing techniques for the agile projects. The results show that WSTP technique improves the effectiveness of test cases prioritization by average of APFD equals to 0.78 for the different parameters. However, the results vary according to the weights given to the parameters and the nature of the sprint, which provides significant implications for the software companies. By prioritizing test cases based on the agile parameters, companies can improve their regression testing process for the agile projects. We also focused on using the priorities of the found issues, not only their count – as mostly done – which provides more accurate prioritization results. In addition, the results show that CRTS technique improves the effectiveness of test cases selection by average of F-measure equals to 0.79 for the different releases with different number of word occurrences applied. We used a clustering technique to deal with the large datasets of test cases to save cost and time. Moreover, we applied text mining to the issues in order to determine the related test cases that test the same functionalities as the failed ones. The significant contribution of this research is

the usage of agile parameters for regression testing of agile projects specifically. Based on our study, these parameters have never been addressed before to tackle this field. Many directions can be extended in the future work, such as: (a) considering different agile parameters to be used for test cases prioritization or selection and (b) merging prioritization and selection techniques at the release level, so that the selected set of test cases is prioritized to identify which of the selected test cases should run first.

REFERENCES

1. Sureshchandra Kalpana, Shrinivasavadhani Jagadish. Moving from Waterfall to Agile. *Agile 2008 conference*; 97–101. DOI: 10.1109/Agile.2008.49.
2. Malhotra C, Chug A. Agile Testing with Scrum – A Survey. *International Journal of Advanced Research in Computer Science and Software Engineering* 2013; **3**(3):452–459.
3. Harish R, Madhu BK, Loksha V. A Sophisticated Study on Best Practices of Agile Software Testing. *International Journal of Electronics Communication and Computer Engineering* 2012; **3**(1):26–30.
4. Harrold MJ, Orso A. Retesting software during development and maintenance. In *ICSM: Frontiers of Software Maintenance*, Sep. 2008; pp. 88–108.
5. Yoo S, Harman M. Regression Testing Minimisation, Selection and Prioritisation – A Survey. *Software Testing verification and reliability* 2009; **22**(2):67–120.
6. Hellmann Theodore D, Sharma Abhishek, Ferreira Jennifer. Agile Testing: Past, Present, and Future – Charting a Systematic Map of Testing in Agile Software Development. *Agile Conference*: 2012; 55 – 63.
7. Parsons D, Susnjak T, Lange M. Influences on regression testing strategies in agile software development environments. *Software Quality Journal* 2014; **22**(4):717–739.
8. Graves TL, Harrold MJ, Kim J, Porter A, Rothermel G. An empirical study of regression test selection techniques. *ACM Transactions on Software Engineering Methodology* 2001; **10**(2):184–208.
9. Talby D, Keren A, Hazzan O, Dubinsky Y. Agile software testing in a large-scale project. *IEEE Software* 2006; **23**(4):30–37.
10. Gittens M, Lutfiyya H, Bauer M, Godwin D, Kim YW, Gupta P. An empirical evaluation of system and regression testing. In *Proceedings of the 2002 conference of the Centre for advanced studies on collaborative research (CASCON '02)*, 2002; 3.
11. Shaye Susan D. Transitioning a team to agile test methods. *Agile conference*, 2008.
12. Downs G. Lean-agile acceptance test-driven development: better software through collaboration by Ken Pugh. *ACM SIGSOFT Software Engineering Notes* 2011; **36**(4):.
13. Miller RW, Collins CT. Acceptance Testing. 2001 *XP Proc. Conference*, 2001.
14. Crispin Lisa, Gregory Janet. Agile testing: a practical guide for testers and agile teams. *Pearson Education*, 2009.
15. Naresh C. Anita. Testing in an agile environment: a new Approach. *International Conf. on Next Generation Communication and Computing System (ICNGCCS-2010)*.
16. Kapfhammer GM. Empirically evaluating regression testing techniques: challenges, solutions, and a potential way forward. *IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*, 2011; 99–102.
17. Ghiduk AS, Girgis MR, Abd-Elkawy EH. A survey of regression testing techniques. *International Journal of Advanced Research in Computer Science* 2012; **3**(5):.
18. Bharati C, Verma S. Analysis of different regression testing approaches. *International Journal of Advanced Research in Computer and Communication Engineering* 2013; **2**(5):2150–2155.
19. Elbaum S, Malishevsky AG, Rothermel G. Test case prioritization: a family of empirical studies. *IEEE Transactions on Software Engineering* 2002; **28**(2):159–182.
20. Engström E, Runeson P, Skoglund M. A systematic review on regression test selection techniques. *Information and Software Technology* 2009; **52**(1):14–30.
21. Smith AM, Kapfhammer GM. An empirical study of incorporating cost into test suite reduction and prioritization. *Proceedings of the 2009 ACM Symposium on Applied Computing - SAC'09*, 2009; 461. DOI:10.1145/1529282.1529382.
22. Clemson M, Gupta R, Jean M, Unwersity. A methodology for controlling the size of a test suite. *ACM Transactions on Software Engineering and Methodology* 1993; **2**(3):270–285. DOI: 10.1109/ICSM.1990.131378.
23. Panigrahi CR, Mall R. A hybrid tegration test selection technique for object-oriented programs. *International Journal of Software Engineering and its Applications* 2012; **6**(4):17–34.
24. Larsen L, Harrold M. Slicing object-oriented software. In *Proceedings of 18th IEEE International, Conference on Software Engineering*, 1996; 495–505.
25. Yoo S, Harman M, Tonella P, Susi A. Clustering test cases to achieve effective & scalable prioritisation incorporating expert knowledge. *ISSA'09 Proceedings of the eighteenth international symposium on Software testing and analysis*, 2009; 201–212.
26. Carlson R, Hyunsook Do, Denton A. A clustering approach to improving test case prioritization. *27th IEEE Inter. Conference on Software Maintenance*, 2011; pp 382–391.
27. Chen S, Chen Z, Zhao Z, Xu B, Feng Y. Using semi-supervised clustering to improve regression test selection techniques. *2011 Fourth IEEE International Conference on Software Testing, Verification and Validation*, 2011; 1–10. DOI:10.1109/ICST.2011.38

28. Zhang DQ, Chen SC, Zhou ZH. Semi-supervised dimensionality reduction. In *Proceedings of 7th SIAM Inter. Conference on Data Mining(SDM '07)*, 2007; 629–634.
29. Bach J. Risk and requirements-based testing. *Computer* 1999; **32**(6):113–114.
30. Muthusamy T. A new effective test case prioritization for regression testing based on prioritization algorithm. *International Journal of Applied Information Systems* 2014; **6**(7):21–26.
31. Raju S, Uma GV. An Efficient Method to Achieve Effective Test Case Prioritization in Regression Testing using Prioritization Factors. *Asian Journal of Information Technology* 2012; **11**(5):169–180. DOI:10.3923/ajit.2012.169.180.
32. Arafeen MJ, Do H. Test case prioritization using requirements-based clustering. *Proceedings - IEEE 6th International Conference on Software Testing, Verification and Validation, ICST* 2013; 312–321, DOI:10.1109/ICST.2013.12.
33. Ural H, Yenigün H. Regression test suite selection using dependence analysis. *Journal of Software: Evolution and Process* 2013; **25**(7):681–709.
34. Anita Arora, Naresh Chauhan. A regression test selection technique by optimizing user stories in an agile environment. In *Advance Computing Conference (IACC), 2014 IEEE International*, 1454–1458.
35. Anita Arora, Naresh Chauhan A. linguistic approach for testcase prioritization in an agile environment. *13th Annual International STC* 2013.
36. Cohn M. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional, 2004.
37. Berkhin P. *Survey of clustering data mining techniques*. Accrue Software, Inc, 2002.
38. He MX, Petoukhov SV, Ricci PE. Genetic code, Hamming distance and stochastic matrices. *Bulletin of Mathematical Biology* 2004; **66**(5):1405–1421.
39. Askarunisa MA, Shanmugapriya ML, Ramaraj DN. Cost and coverage metrics for measuring the effectiveness of test case prioritization techniques. *INFOCOMP Journal of Computer Science* 2010; **9**(1):43.
40. Elbaum SG, Malishevsky AG, Rothermel G. Prioritizing test cases for regression testing. *Proceedings of International Symposium on Software Testing and Analysis (ISSTA 2000)*, ACM Press, 2000; 102–112.